Lab Guide: Building the School Portal Skeleton

Course: Advance Web

Lab Session: Class 2 (2 Hours)

🎯 Learning Objectives

- Create and register a new Django App.
- Configure a two-tier URL routing system.
- Write Function-Based Views (FBV).
- Render HTML templates and handle dynamic URL parameters.

Part 1: App Creation & Registration (20 Mins)

Every new feature in Django starts with a "startapp" command to maintain modularity.

1. **Open your terminal** and navigate to your project folder (e.g., mysite).
2. **Create the app**:

   Bash

   ```
   python manage.py startapp students
   ```

3. **Register the app**: Open `mysite/settings.py`, find the `INSTALLED_APPS` list, and add `'students',` at the bottom.

   > **Theory Check:** Why register? Django ignores any app directory unless it is explicitly listed in `settings.py`.

Part 2: The Logic Layer - Your First View (25 Mins)

The **View** is a Python function that processes a request and returns a response.

1. Open `students/views.py`.
2. **Write the home view**:

   Python

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome to the School Portal!")
```

Part 3: The Mapping Layer - URL Routing (35 Mins)

To see your view in a browser, you must map it to a URL path.

1. **App-Level Routing**: Inside the `students` folder, create a new file named `urls.py`.
2. **Add the routing code**:

   Python

   ```
   from django.urls import path
   from . import views

   urlpatterns = [
       path('', views.home, name='students_home'),
   ]
   ```

3. **Project-Level Routing**: Open `mysite/urls.py` and connect the app.

   Python

   ```
   from django.urls import path, include

   urlpatterns = [
       path('admin/', admin.site.urls),
       path('students/', include('students.urls')), # Connects app
   URLs
   ]
   ```

Part 4: The Presentation Layer - Templates (20 Mins)

Instead of returning plain text, we will now return a real HTML file.

1. **Create the Directory**: Inside the `students` folder, create this exact folder path: `templates/students/`.
2. **Create the File**: Inside that folder, create `home.html`.
3. **Add HTML Content**:

   HTML

   ```
   <!DOCTYPE html>
   ```

```
<html>
<body>
    <h1>Welcome to the School Portal</h1>
    <p>This is the students app home page.</p>
</body>
</html>
```

4. **Update the View**: Go back to `students/views.py` and change the return statement:

   Python

   ```
   from django.shortcuts import render

   def home(request):
       return render(request, 'students/home.html')
   ```

Part 5: Challenge - Dynamic Student IDs (20 Mins)

Now, create a page that changes based on the URL (e.g., `/students/5/`).

1. **Create the Detail View** in `students/views.py`:

   Python

   ```
   def student_detail(request, student_id):
       return HttpResponse(f"Student Profile Page - ID:
   {student_id}")
   ```

2. **Add the Dynamic URL** in `students/urls.py`:

   Python

   ```
   path('<int:student_id>/', views.student_detail,
   name='student_detail'),
   ```

3. **Test it**: Run `python manage.py runserver` and visit `http://127.0.0.1:8000/students/7/`.

✅ Checklist for Completion

- [ ] Does `http://127.0.0.1:8000/students/` show your HTML page?
- [ ] Does `http://127.0.0.1:8000/students/10/` show "ID: 10"?